```
var_dump(filter_var(
    '123.4',
    FILTER_SANITIZE_NUMBER_FLOAT,
    FILTER_FLAG_ALLOW_FRACTION
));
```

```
Output:
string(5) "123.4"
```

… but why this isn't the default behavior is beyond me.

`FILTER_SANITIZE_SPECIAL_CHARS` and `FILTER_SANITIZE_FULL_SPECIAL_CHARS` function like `htmlspecialchars()` and `htmlentities()`, respectively, and it is recommended that you just call those functions rather than going round the houses with `filter_var()`. Nevertheless, here they are at work:

```
$str = '"&\'<>£¥©®';

var_dump(filter_var($str, FILTER_SANITIZE_SPECIAL_CHARS));
var_dump(filter_var($str, FILTER_SANITIZE_FULL_SPECIAL_CHARS));
```

```
Output:
string(33) "&#34;&#38;&#39;&#60;&#62;£¥©®"
string(48) "&quot;&&#039;&lt;&gt;&pound;&yen;&copy;&reg;"
```

I do not recommend touching `FILTER_SANITIZE_EMAIL` or `FILTER_SANITIZE_URL` at all. Both will destroy any non-Latin UTF-8 characters, which are used increasingly in URLs in this brave new modern era. Accented characters have been permitted in the email address specification for a while now and are quite common even in English-speaking regions. You don't want to annoy any colleagues or customers with Irish or Scottish heritage now, do you?

We can wrap up this quick delve into the `filter` extension here. I think I've made it clear that for all its utility, it does have a lot of pitfalls, so tread carefully when using it. I realize that "don't use the URL or email functionality" isn't particularly helpful if validating/sanitizing those patterns is a problem you need to solve, so in the spirit of being helpful: emails can be validated by an RFC 6532 compliant pattern or library such